

MySQL 面试题

1. 基础

问：执行一条 SQL 查询语句，期间发生了什么？

答：

连接器（权限校验）→ 分析器（词法/语法）→ 优化器（选索引/执行计划）
→ 执行器（调用存储引擎接口）→ 存储引擎（读写数据）

MySQL 8.0 已移除查询缓存。

问：InnoDB 和 MyISAM 的区别？

答：

特性	InnoDB	MyISAM
事务	支持	不支持
行锁	支持	表锁
外键	支持	不支持
崩溃恢复	redo log	不支持
MVCC	支持	不支持
默认	MySQL 5.5+	已淘汰

问：MySQL 数据文件有哪些？

答：

文件	说明
<code>.ibd</code>	InnoDB 表空间（数据+索引）
<code>.frm</code> （8.0 前）	表结构定义
<code>ibdata</code>	系统表空间（undo、double write buffer 等）
<code>redo log</code>	redo 日志（ <code>ib_logfile</code> ）
<code>undo log</code>	undo 段
<code>binlog</code>	Server 层归档日志

问：数据库三大范式是什么？

答：

范式	要求
1NF	列不可再分（原子性）
2NF	1NF + 非主键列完全依赖主键（消除部分依赖）
3NF	2NF + 非主键列不依赖其他非主键列（消除传递依赖）

反范式：为性能适当冗余（如订单存用户名），用空间换时间。

2. 索引（最高频）

问：为什么 MySQL 用 B+ 树而不是 B 树或 Hash？

答：– vs B 树：B+ 树非叶子节点只存索引，一页存更多 key，树更矮，IO 更少；叶子节点链表便于范围查询 – vs Hash：Hash 只支持等值查询，不支持范围查询和排序 – vs 二叉树：树太高，IO 次数多

问：聚簇索引和非聚簇索引的区别？

答：– 聚簇索引（主键索引）：叶子节点存完整行数据，一张表只有一个 – 二级索引（辅助索引）：叶子节点存主键值，查询需回表

覆盖索引：查询列全在索引中，无需回表，Extra 显示 `Using index`。

问：联合索引的最左匹配原则？

答：索引 `(a, b, c)` 相当于创建了 `(a)`、`(a,b)`、`(a,b,c)` 三个索引。

- 能用：`WHERE a=1`、`WHERE a=1 AND b=2`、`WHERE a=1 AND b=2 AND c=3`
- 不能用：`WHERE b=1`、`WHERE c=1`、`WHERE b=1 AND c=2`

问：索引失效的场景？

答：1. 对索引列使用函数或运算：WHERE YEAR(create_time)=2024 2. 隐式类型转换：WHERE phone = 13800138000 (phone 是 varchar) 3. 左模糊：LIKE '%abc' 4. OR 一侧无索引 5. 违反最左匹配 6. 优化器判断全表扫描更快（数据量小） 7. !=、<>、NOT IN 可能导致失效

问：索引有哪些分类？索引是不是越多越好？

答：

分类	类型
数据结构	B+ 树（默认）、Hash（Memory 引擎）
物理	聚簇索引、二级索引
字段	主键、唯一、普通、前缀、联合、覆盖
逻辑	单列、联合

不是越多越好： – 占用磁盘和内存 – 写操作需维护索引，降低 INSERT/UPDATE/DELETE 性能 – 优化器选择困难

低区分度字段（如性别 status 0/1）不适合单独建索引，优化器可能全表扫描更快。

问：EXPLAIN 怎么看？type 和 Extra 重点字段？

答：

```
EXPLAIN SELECT * FROM user WHERE id = 100;
```

字段	说明
type	访问类型，性能：system > const > eq_ref > ref > range > index > ALL
key	实际使用的索引，NULL 表示未走索引
rows	预估扫描行数
Extra	Using index 覆盖索引；Using filesort 需优化；Using temporary 用临时表

干预索引：FORCE INDEX、USE INDEX、IGNORE INDEX（优化器选错时用）。

问：IN 和 EXISTS 怎么选？

答：

场景	推荐
外表小、内表大	IN（外表驱动）
外表大、内表小	EXISTS（内表驱动，找到即返回）

本质都是嵌套循环，优化器可能转为 semi-join。能写 JOIN 时优先 JOIN。

3. 事务

问：ACID 怎么实现？

答：

特性	实现
原子性 (A)	undo log 回滚
一致性 (C)	业务 + AID 共同保证
隔离性 (I)	锁 + MVCC
持久性 (D)	redo log

问：四种隔离级别及问题？

答：

级别	脏读	不可重复读	幻读
读未提交	✓	✓	✓
读已提交 (RC)	✗	✓	✓
可重复读 (RR, 默认)	✗	✗	部分解决
串行化	✗	✗	✗

- **脏读**：读到未提交的数据
- **不可重复读**：同一事务两次读同一行，结果不同（被其他事务修改）
- **幻读**：同一事务两次范围查询，行数不同（被其他事务插入/删除）

问：MVCC 是怎么实现的？

答：多版本并发控制，通过 隐藏列 + undo log 版本链 + Read View 实现无锁读。

隐藏列： - `trx_id`：最后修改的事务 ID - `roll_pointer`：回滚指针，指向 undo log 中的旧版本

Read View 四个字段： - `creator_trx_id`：创建者事务 ID - `m_ids`：活跃事务 ID 列表 - `min_trx_id`：最小活跃事务 ID - `max_trx_id`：下一个事务 ID

可见性判断：从版本链头部遍历，找到对当前事务可见的版本。

- RC：每次 SELECT 生成新 Read View
- RR：首次 SELECT 生成 Read View，之后复用

问：MySQL 可重复读完全解决幻读了吗？

答：没有完全解决，但通过两种方式很大程度避免：

1. 快照读（普通 SELECT）：MVCC 保证看到一致性快照
2. 当前读（SELECT FOR UPDATE、INSERT、UPDATE、DELETE）：间隙锁 + 临键锁 阻止其他事务插入

4. 锁

问：MySQL 有哪些锁？

答：

分类	类型
粒度	全局锁、表锁、行锁
模式	共享锁 (S)、排他锁 (X)
行锁类型	记录锁、间隙锁、临键锁（记录+间隙）
意向锁	IS、IX（表级，与行锁兼容）

问：update 没加索引会锁全表吗？

答：会。WHERE 条件未命中索引时，InnoDB 对所有行加 next-key lock，等效锁全表。生产环境务必保证 WHERE 条件走索引。

问：两个事务更新不同主键范围会阻塞吗？

答：走索引时：WHERE id < 10 和 WHERE id > 15 锁不同记录/间隙，一般不阻塞。

不走索引时：全表扫描，对所有行加 next-key lock，会互相阻塞。

间隙锁示例：WHERE id > 5 AND id < 10 FOR UPDATE 锁 (5,10) 间隙，阻止插入 id=6~9。

5. 日志（必考）

问：redo log、undo log、binlog 的区别？

答：

日志	层次	作用
redo log	InnoDB 引擎层	崩溃恢复，保证持久性（WAL）
undo log	InnoDB 引擎层	事务回滚、MVCC 多版本
binlog	Server 层	主从复制、数据恢复

问：为什么需要两阶段提交？

答：保证 redo log 和 binlog 一致性。流程：

1. 写 undo log
2. 更新 Buffer Pool
3. 写 redo log (prepare)
4. 写 binlog
5. 提交 redo log (commit)

崩溃恢复时：– redo prepare + binlog 完整 → 提交 – redo prepare + binlog 不完整 → 回滚

问：redo log 写在哪里？为什么需要 double write buffer？

答：redo log：先写 redo log buffer（内存），fsync 刷到磁盘 redo log 文件（循环写）。

为什么先写 redo 不直接写数据页：– WAL：顺序写日志比随机写数据页快 – 崩溃恢复时重做已提交事务

double write buffer（双写缓冲）：– 问题：数据页 16KB，OS 页 4KB，刷盘非原子，可能 partial page write（页损坏）– redo 无法修复已损坏的页（只记录「改了什么」，页本身坏了无法应用）– 解决：页先写到 DWB（共享表空间顺序写）→ 再写 .ibd 随机写；崩溃时从 DWB 恢复完整页副本

6. 架构与优化

问：主从复制原理？

答：

```
主库：事务提交 → 写 binlog
从库 IO 线程：拉取 binlog → 写 relay log
从库 SQL 线程：重放 relay log
```

延迟原因：从库单线程回放、大事务、网络延迟。

问：分库和分表有什么区别？

答：

对比	分库	分表
目的	解决连接数/IO 瓶颈	解决单表数据量过大
方式	水平/垂直拆到多个库	同库拆多表或分库+分表
复杂度	跨库 JOIN 困难	相对简单

常见方案： – 垂直分库：按业务拆（订单库、用户库） – 垂直分表：大表拆字段（主表+扩展表） – 水平分表：按 user_id % N 分表 – 水平分库：分表后再分散到不同实例

中间件：ShardingSphere、MyCat。

问：慢 SQL 怎么优化？

答：1. EXPLAIN 分析：看 type、key、rows、Extra 2. 加索引 / 优化索引 3. 避免 SELECT * 4. 分页优化（延迟关联、游标分页） 5. 拆分大 SQL 6. 读写分离

7. SQL 执行详细流程

问：执行一条 UPDATE 语句，InnoDB 内部详细流程？

答：

1. 连接器：验证权限，建立连接（长连接注意 max_connections）
2. 分析器：词法分析 → 语法分析，生成语法树
3. 优化器：选择索引、确定执行计划（是否走索引、join 顺序）
4. 执行器：
 - a. 调用存储引擎接口「取第一行」
 - b. 存储引擎从 Buffer Pool 或磁盘读数据页
 - c. 执行器判断 WHERE 条件，满足则更新
 - d. 存储引擎更新 Buffer Pool 中的页（标记为脏页）
 - e. 写 undo log（旧值，供回滚和 MVCC）
 - f. 写 redo log（prepare 状态，WAL 先日志后数据）
 - g. 写 binlog
 - h. 提交 redo log（commit）
 - i. 后台线程异步刷脏页到磁盘
5. 返回客户端 affected rows

关键：数据修改先写 Buffer Pool 和 redo log，磁盘数据页异步刷盘。

问：SQL 语句的执行顺序是什么？

答：

```
SELECT DISTINCT column, aggregate_func
FROM table
JOIN other ON ...
WHERE condition
GROUP BY column
HAVING aggregate_condition
ORDER BY column
LIMIT n;
```

执行顺序：

```
FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT
```

记忆：从表找数据 → 过滤 → 分组 → 过滤组 → 选列 → 排序 → 分页。

8. 幻读与 ReadView

问：举一个幻读的具体例子？

答：事务 A（RR 隔离级别）：

```
-- T1: 事务 A 开始
BEGIN;
SELECT * FROM orders WHERE amount > 100; -- 返回 5 条

-- T2: 事务 B 插入一条 amount=200 的记录并提交
INSERT INTO orders (amount) VALUES (200);
COMMIT;

-- T3: 事务 A 再次查询
SELECT * FROM orders WHERE amount > 100; -- 快照读仍返回 5 条 (MVCC)

-- T4: 事务 A 当前读
SELECT * FROM orders WHERE amount > 100 FOR UPDATE; -- 返回 6 条!
```

幻读：同一事务内，相同范围查询，**当前读** 结果行数不同。快照读靠 MVCC 避免，当前读靠间隙锁避免。

问：ReadView 可见性判断规则详解？

答：从版本链最新版本开始，对每个版本的 `trx_id` 判断：

1. `trx_id == creator_trx_id` → 可见（自己修改的）
2. `trx_id < min_trx_id` → 可见（事务已提交）
3. `trx_id >= max_trx_id` → 不可见（未来事务）
4. `min_trx_id ≤ trx_id < max_trx_id`：
 - `trx_id` 在 `m_ids` 中 → 不可见（活跃未提交）

- `trx_id` 不在 `m_ids` 中 → 可见（已提交）

不可见则沿 `roll_pointer` 找旧版本，直到找到可见版本或链尾（返回空）。

RC vs RR: RC 每次 SELECT 新建 ReadView; RR 第一次 SELECT 创建后复用，所以 RR 不可重复读也避免了。

9. Buffer Pool 与 change buffer

问：Buffer Pool 是什么？有什么作用？

答：InnoDB 在内存中的 **缓存区域**，缓存： – 数据页（索引页） – 插入缓冲（change buffer） – 自适应哈希索引 – 锁信息

作用：减少磁盘 IO，读写先在 Buffer Pool 操作。

LRU 优化：分为 Young 区（5/8）和 Old 区（3/8），新读入的页先放 Old 区头部，避免全表扫描污染热点数据。

参数：`innodb_buffer_pool_size`（建议物理内存 60~80%）。

问：什么是 change buffer（写缓冲）？

答：当更新 **非唯一二级索引** 页时，若目标页不在 Buffer Pool 中，不立即读磁盘，而是将变更记录到 **change buffer**（内存）。

后续：该页被读入 Buffer Pool 时，**merge** change buffer 中的变更。

优点：减少随机 IO，提升写性能。

限制： – 仅适用于 **非唯一二级索引**（唯一索引需立即读页判重） – 对应数据页已在 Buffer Pool 则直接更新，不写 change buffer

持久化：change buffer 也写 redo log，崩溃可恢复。

10. 主从延迟

问：MySQL 主从延迟的原因和解决方案？

答：原因： 1. 从库 **单线程** 回放 relay log（MySQL 5.6 前），主库多线程写入 2. **大事务**：一个事务 binlog 很大，回放耗时长 3. **网络延迟** 4. 从库硬件/负载差 5. 从库做备份、统计等额外负载

解决方案：

方案	说明
并行复制	MySQL 5.7+ <code>slave_parallel_workers</code> 按库/组提交并行
读写分离策略	写后立即读走主库，或关键读走主库
缓存	写入后缓存，读先查缓存
半同步复制	至少一个从库确认收到 binlog 才返回 (<code>rpl_semi_sync_master</code>)
监控告警	<code>Seconds_Behind_Master</code> 监控延迟

11. 慢 SQL 优化案例

问：一个典型的慢 SQL 优化案例？

答：原始 SQL：

```
SELECT * FROM orders
WHERE user_id = 123 AND status = 1
ORDER BY create_time DESC
LIMIT 10000, 20;
```

问题：1. `SELECT *` 回表开销大 2. 深分页 `LIMIT 10000, 20` 需扫描 10020 行 3. 可能缺少合适联合索引
优化：

```
-- 1. 联合索引覆盖
ALTER TABLE orders ADD INDEX idx_user_status_time (user_id, status, create_time);

-- 2. 延迟关联
SELECT o.* FROM orders o
INNER JOIN (
    SELECT id FROM orders
    WHERE user_id = 123 AND status = 1
    ORDER BY create_time DESC
    LIMIT 10000, 20
) t ON o.id = t.id;

-- 3. 游标分页 (推荐)
SELECT * FROM orders
WHERE user_id = 123 AND status = 1 AND id < last_id
ORDER BY id DESC LIMIT 20;
```

EXPLAIN 关注：type 至少 range，Extra 避免 Using filesort。

12. 前缀索引

问：什么是前缀索引？有什么优缺点？

答：对长字符串列，只索引前 N 个字符：

```
ALTER TABLE user ADD INDEX idx_email (email(10));
```

优点：减少索引空间，提升写入性能。

缺点： 1. 无法 覆盖索引（需完整列） 2. 无法 ORDER BY 该列 3. 可能 区分度不够（前缀相同的多）

选择长度：逐步增加 N，使区分度接近全列索引：

```
SELECT COUNT(DISTINCT LEFT(email, N)) / COUNT(*) FROM user;
```

一般区分度 > 95% 即可。

13. 其他深入

问：count(*)、count(1)、count(列) 的区别？

答：InnoDB 中三者性能几乎相同，优化器会优化： – count(*)：统计行数，**推荐** – count(1)：统计行数，不读列值 – count(列)：统计列非 NULL 行数，需判断 NULL

MyISAM 无 WHERE 时 count(*) 直接读表行数，极快。

大表优化：Redis 计数、汇总表、近似 count（HyperLogLog）。

问：varchar 和 char 的区别？如何选择？

答：

对比	char	varchar
长度	固定，不足补空格	可变，存长度前缀
空间	可能浪费	节省
性能	略快（无长度计算）	略慢
场景	定长（手机号 char(11)、MD5 char(32)）	变长（姓名、地址）

图解加深

1. 一条 SQL 的 InnoDB 视角

理解要点：Buffer Pool → 数据页 → redo log prepare → binlog → redo commit → 异步刷盘。

2. MVCC 版本链

一行记录被多次修改，roll_pointer 串成链表，ReadView 从链头遍历找可见版本。RC 每次新建 ReadView，RR 复用。

3. 间隙锁范围

WHERE id > 5 AND id < 10 的 FOR UPDATE 会锁 (5, 10) 间隙，阻止插入 id=6,7,8,9。

大厂追问

1. 为什么 RR 是默认隔离级别？

历史原因 + 平衡：比 RC 多避免不可重复读，性能比串行化好。配合 MVCC + 间隙锁，大部分幻读也避免了。

2. redo log 和 binlog 为什么两个都要？

redo log 是 InnoDB 引擎层崩溃恢复（循环写，固定大小）；binlog 是 Server 层主从复制和数据归档（追加写）。两阶段提交保证两者一致。

3. 索引下推（ICP）是什么？

MySQL 5.6+，联合索引 (a, b) 查询 WHERE a=1 AND b=2，在存储引擎层直接过滤 b，减少回表。Extra 显示 Using index condition。

4. 为什么不建议用 UUID 做主键？

UUID 随机，插入 B+ 树导致大量页分裂和碎片，写性能差。推荐自增 ID 或有序 UUID（如 snowflake）。

5. 如何定位慢 SQL？

开启 slow_query_log，long_query_time=1；用 pt-query-digest 分析；EXPLAIN + 执行计划；监控 Performance Schema。

6. binlog 三种格式区别？

Statement：记 SQL，日志小，可能主从不一致（now()、UUID）。**Row**：记行变更，安全，日志大。

Mixed：默认 Statement，不安全时转 Row。推荐 **Row**。

7. 串行化隔离级别如何实现？

对所有读加 **共享锁**，读写互斥，完全串行，性能最差，一般不用。

8. 一条 UPDATE 是原子性的吗？

是。InnoDB 事务保证，要么全部成功要么全部回滚（undo log），即使更新多行也是一条原子操作。